

Frontend Proje El Kitabı (JS/HTML/CSS) — Senior Seviye Standartlar

Sürüm: 1.0 • Tarih: 29 Ekim 2025 • Hazırlayan: ChatGPT (Senior Frontend rolüyle)

Bu doküman; HTML, CSS ve Vanilla JavaScript (ESM) ağırlıklı bir **frontend proje başlangıç kılavuzu** ve **standartlar seti** sunar. Hedef; tek sayfa (SPA/MPA fark etmeksizin) projelerde **tutarlılık**, **ölçeklenebilirlik**, **performans**, **erişilebilirlik** ve **bakım kolaylığı** sağlamaktır. Doküman; dizin yapısı, kod standartları, bileşen tasarımı, CSS mimarisi, erişilebilirlik, SEO, performans bütçesi, test, otomasyon, CI/CD ve güvenlik gibi başlıklarda net yönergeler içerir.

İçindekiler

1. Proje Dizini ve Yapı
2. HTML Standartları
3. CSS Mimarisi ve Rehber
4. JavaScript Standartları ve Desenler
5. Bileşen Sistemi (UI Kit)
6. Erişilebilirlik (A11y) Kontrol Listesi
7. Performans ve Kalite Hedefleri
8. SEO ve Metadata
9. Test, Linting ve Formatlama
10. Tooling ve Geliştirme Akışı
11. Git, Branch Stratejisi ve Commit Kuralları
12. Güvenlik (Security) İpuçları
13. Uluslararasılaşdırma (i18n)
14. Analitik ve İzleme
15. Kod İnceleme (PR) Kontrol Listesi
16. Örnek Şablon Dosyalar (.editorconfig, .browserslistrc vb.)
17. Örnek Bileşen (HTML/CSS/JS)
18. Yayına Alma (Deployment) ve Bakım Notları

1. Proje Dizini ve Yapı

Önerilen minimal yapı (build aracı olmadan da çalışabilir; isterseniz Vite ile güçlendirilebilir):

```
project-root/
├── public/                # statik dosyalar (favicon, og-image, robots.txt, fonts/)
├── robots.txt
├── favicon.ico
├── og-image.png
├── src/
│   ├── assets/
│   │   ├── images/
│   │   ├── icons/        # SVG sprite veya ayrı SVG'ler
│   │   └── styles/       # global.css, tokens.css, utilities.css
│   ├── components/      # UI bileşenleri (her bileşen için .html/.css/.js)
│   ├── pages/           # sayfa düzenleri (home.html, about.html vs.)
│   ├── app.js           # giriş (ESM), router/bootstrapping
│   └── app.css          # sayfa genel stilleri (import ile parçalanabilir)
├── .editorconfig
├── .browserslistrc
├── .eslintrc.json
├── .stylelintrc.json
├── .prettierrc
├── index.html           # kök
└── package.json
```

Not: Bileşenleri **başlımsız** ve tekrar kullanılabilir tasarlayın. Her bileşenin kendi HTML/CSS/JS dosyası olmalı; stiller mümkünse kapsüllenmiş olmalı.

2. HTML Standartları

- Semantik** etiketleri kullanın: <header>, <nav>, <main>, <section>, <article>, <aside>, <footer>.
- Her sayfada **tek bir <h1>**; hiyerarşik <h2>, <h3> ... ile devam etmeli.
- Görseller için **alt** ve **width/height** belirtin; CLS'yi azaltır.
- Formlar için label, aria-* ve hata mesajları zorunlu.
- Link buton olması; eylemse <button>, gezinmeyse <a>.
- Meta etiketleri: charset, viewport, description, OpenGraph/Twitter Cards, theme-color.

Örnek temel HTML:

```
<!doctype html>
<html lang="tr">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <meta name="description" content="Proje açıklaması.">
    <meta property="og:title" content="Proje Başlığı">
    <title>Ana Sayfa</title>
    <link rel="preconnect" href="https://fonts.googleapis.com">
  </head>
  <body>
    <header role="banner">...</header>
    <main id="main-content">...</main>
    <footer role="contentinfo">...</footer>
    <script type="module" src="/src/app.js"></script>
  </body>
```

</html>

3. CSS Mimarisi ve Rehber

CSS için **token tabanı** yaklaşımları önerilir: renkler, boşluklar, tipografi ve gölgeler `:root` deklaratifleri ile merkezi tanımlanır. Bileşen stillerini küçük, izole ve **BEM** uyumlu tutun. Yardımcı (utility) sınıfların `utilities.css` içinde toplayın.

Örnek token seti:

```
:root{
  --color-bg:#0b0f14;
  --color-fg:#e6edf3;
  --color-primary:#2d8cff;
  --space-1:4px; --space-2:8px; --space-3:12px; --space-4:16px; --space-6:24px;
  --radius-2:8px; --radius-3:12px;
  --font-sans: ui-sans-serif, system-ui, -apple-system, "Inter", "Segoe UI", Roboto, Arial, "Noto Sans", sans-serif;
}
```

- **BEM**: `.card`, `.card__title`, `.card--featured`.
- **Katmanlama**: `base` (reset/normalize), `tokens`, `utilities`, `components`, `pages`.
- **Responsive**: Mobil-öncelikli; `min-width` breakpoint'leri (480/768/1024/1280).
- **Dark/Light**: `:root` ve `[data-theme="dark"]` deklaratifleri ile tema.
- **Performans**: Özelleştirilmiş font subsetleri, `font-display: swap`.
- **Critical CSS**: Above-the-fold stilleri inline; geri kalanı dinamik yükleyin.

4. JavaScript Standartları ve Desenler

- **ESM** kullanın: `type="module"`, `import/export`.
- **Saf Fonksiyonlar** ve **küçük modüller**: Tek sorumluluk ilkesi.
- DOM erişimlerini **delege** edin; gereksiz reflow/paint tetiklemeyin.
- **Fetch** + **AbortController**; hataları merkezi yönetin.
- **CustomEvent** ile bileşen iletişimi.
- **State**: Küçük projelerde tek dosya store; büyüklerde basit observer pattern.
- **Lint/Format**: ESLint + Prettier zorunlu.

Örnek giriş dosyası:

```
// src/app.js
import { router } from '../lib/router.js';
import { initUI } from '../lib/ui.js';

window.addEventListener('DOMContentLoaded', () => {
  initUI();
  router();
});
```

5. Bileşen Sistemi (UI Kit)

Bileşenler self-contained olmalı: `button.html`, `button.css`, `button.js`. API yüzeyi net; props veri-attributeleri ile tanımlanabilir (`data-variant`, `data-size`).

6. Erişilebilirlik (A11y) Kontrol Listesi

- Kontrast oran: Minimum 4.5:1 (metin) — WCAG AA.
- Klavye ile gezinme: Tüm etkileşimler `Tab` ve `Enter/Space` ile yapılabilmeli.
- `focus` görünür olmalı; outline kapatılmamalı.
- Landmark'lar: `header`, `nav`, `main`, `footer` kullan.
- `aria-live` bölgeleri ve skip link ekle.

7. Performans ve Kalite Hedefleri

- Lighthouse hedefleri: Performans ≥ 90 , Erişilebilirlik ≥ 95 , Best Practices ≥ 95 , SEO ≥ 90 .
- **Bütçeler**: İlk yük JS ≤ 150 KB gzip; CSS ≤ 50 KB gzip; Görseller modern format (AVIF/WEBP).
- **Lazy**: Görseller `loading="lazy"`, `decoding="async"`; modüller `import()` ile ayrıştırılma.

8. SEO ve Metadata

- Her sayfada benzersiz **title** (50–60 karakter) ve **description** (150–160).
- `canonical` ve OpenGraph/Twitter meta'ları.
- Yapısal veri (JSON-LD) — BreadCrumb, Organization, Product vb.

9. Test, Linting ve Formatlama

- **Unit**: Jest/Vitest ile küçük fonksiyonlar.
- **E2E**: Playwright ile kritik akışlar (navigasyon, form gönderimi).
- **Lint**: ESLint (airbnb-base ya da standard), **Stylelint** ve **Prettier** zorunlu.
- **Husky** + **lint-staged** ile pre-commit kontrolü.

10. Tooling ve Geliştirme Akışı

Önerilen paketler: `vite`, `eslint`, `stylelint`, `prettier`, `husky`, `lint-staged`, `postcss`, `autoprefixer`.
Komutlar: `dev`, `build`, `preview`, `test`, `lint`, `format`.

11. Git, Branch Stratejisi ve Commit Kuralları

- Branch modeli: `main` (korunmalı), `develop`, `feature/\*`, `fix/\*`, `release/\*`.
- Commit formatı (Conventional Commits): `feat:`, `fix:`, `docs:`, `refactor:`, `style:`, `test:`, `build:`, `ci:`.
- PR gereksinimi: 1 onay, lint/test geçer, ekran görüntüsü/önizleme linki.

12. Güvenlik (Security) Riskleri

- DOM'a enjekte edilen verileri **escape** edin; `textContent` tercih edin.
- `rel="noopener noreferrer"` dış linklerde zorunlu.
- Üçüncü parti scriptleri minimumda tutun; `integrity` ve `referrerpolicy` kullanın.
- Yerel depolama/çerezlere hassas veri koymayın.

13. Uluslararasılaştırma (i18n)

- Metinleri `src/locales/{tr,en}/common.json` altında toplayın.
- Tarih/sayma biçimlendirme için Intl API kullanın.
- RTL dilleri için `dir="rtl"` desteği.

14. Analitik ve İzleme

- Privacy-first analitik (örn. Plausible/Umami) ya da GA4.
- Hata/RUM: Sentry gibi bir izleme aracı.

15. Kod İnceleme (PR) Kontrol Listesi

- [] Lint + test geçti mi?
- [] Erişilebilirlik ve klavye desteği var mı?
- [] Responsive tüm kırılma noktalarında test edildi mi?
- [] Performans bütçesine uyuyor mu?
- [] Anlamlı ekran görüntüsü/önizleme linki eklendi mi?

16. Örnek Şablon/Config Dosyaları

.editorconfig

```
# .editorconfig
root = true
[*]
charset = utf-8
end_of_line = lf
indent_style = space
indent_size = 2
insert_final_newline = true
trim_trailing_whitespace = true
```

.browserslistrc

```
# .browserslistrc
> 0.5%
last 2 versions
not dead
```

.eslintrc.json

```
{
  "env": { "browser": true, "es2022": true },
  "extends": ["eslint:recommended", "plugin:import/recommended"],
  "parserOptions": { "ecmaVersion": "latest", "sourceType": "module" },
  "rules": {
    "no-unused-vars": ["warn", { "argsIgnorePattern": "^_" }],
    "no-console": ["warn", { "allow": ["warn", "error"] }],
    "import/order": ["error", { "alphabetize": { "order": "asc" } }]
  }
}
```

.stylelintrc.json

```
{
  "extends": ["stylelint-config-standard"],
  "rules": {
    "color-hex-length": "short",
    "block-no-empty": true
  }
}
```

```
}  
}
```

.prettierrc

```
{  
  "printWidth": 100,  
  "singleQuote": true,  
  "semi": true,  
  "trailingComma": "es5"  
}
```

package.json

```
{  
  "name": "frontend-starter",  
  "private": true,  
  "type": "module",  
  "scripts": {  
    "dev": "vite",  
    "build": "vite build",  
    "preview": "vite preview",  
    "lint": "eslint . && stylelint \"**/*.css\"",  
    "format": "prettier --write .",  
    "test": "vitest"  
  },  
  "devDependencies": {  
    "vite": "^5.0.0",  
    "eslint": "^9.0.0",  
    "stylelint": "^16.0.0",  
    "prettier": "^3.0.0",  
    "vitest": "^2.0.0",  
    "husky": "^9.0.0",  
    "lint-staged": "^15.0.0",  
    "autoprefixer": "^10.4.0",  
    "postcss": "^8.4.0"  
  }  
}
```

17. Örnek Bileşen (Card)

card.html

```
<!-- src/components/card/card.html -->  
<article class="card card--featured" role="article">  
    
  <div class="card__body">  
    <h3 class="card__title">Başlık</h3>  
    <p class="card__desc">Kısa açıklama...</p>  
    <button class="card__action" type="button">İncele</button>  
  </div>  
</article>
```

card.css

```
/* src/components/card/card.css */  
.card{background:#111;color:#e6edf3;border-radius:12px;padding:16px;display:grid;gap:12px;box-shadow:0 8px 20px rgba(0,0,0,.25)}  
.card__media{border-radius:8px;display:block;max-width:100%;height:auto}
```

```
.card__title{font-weight:700;margin:0}
.card__desc{opacity:.9;margin:0}
.card__action{background:var(--color-primary);color:#fff;border:0;border-radius:10px;padding:10px 14px;cursor:pointer}
.card--featured{outline:1px solid rgba(45,140,255,.35)}
@media (min-width:768px){.card{grid-template-columns:1fr 1.2fr;align-items:center}}
```

card.js

```
// src/components/card/card.js
export function mountCard(root){
  const action = root.querySelector('.card__action');
  const controller = new AbortController();
  action.addEventListener('click', () => {
    root.dispatchEvent(new CustomEvent('card:inspect', { bubbles:true, detail:{ id: 'example' } }));
  }, { signal: controller.signal });
  return () => controller.abort();
}
```

18. Yayına Alma (Deployment) ve Barındırma Notları

- Statik siteler için: Vercel, Netlify veya Cloudflare Pages.
- Cache stratejisi: `Cache-Control` header'ları (HTML düşük, assets uzun).
- `404.html` ve `\_redirects` (SPA için) ayarları.
- Ortamlar: dev → staging → prod; sürüm numarası ve deployment günlüğü.

Bu el kitabını projeye `docs/` klasörü altında eklemenizi ve PR süreçlerinde referans almanızı öneririm.